

N° solicitud: #16

Fecha de emisión: 27 de mayo de 2026

Estudiante

Nombre: Brandon Pérez

Correo institucional: brandon.perez@est.univalle.edu

Carrera: Ingeniería de Sistemas

Registrado por Unidad de Trámites: Ana Rojas Fernández

#	Trámite / procedimiento	Referencia costo
1	Cambio de Carrera	Bs. 100.00

Observaciones:

Constancia final con registro de firmas de responsables institucionales (control documental).

Este documento consolida el registro de firmas correspondientes al flujo académico-administrativo (cambio de carrera / convalidación interna u homólogos). Las imágenes adjuntas corresponden a las rúbricas institucionales cargadas por Trámites.



Puede comprobar la autenticidad de esta constancia
escaneando el QR o con el código en el portal institucional.

Cód.: UV-2026-CON-16-2AA0CD47

Control de firmas — responsables institucionales

Trámite N° 16 · 27 de mayo de 2026

Director de carrera (origen)
V°B° y análisis de solicitud / informe según procedimiento



Firma

Director de carrera (destino)
Informe técnico y convalidación (equivalencia >= 70%)



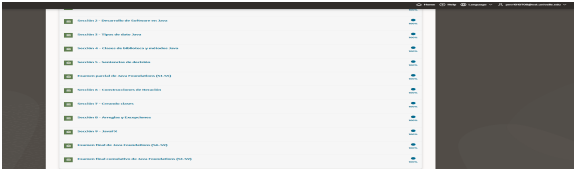
Firma

Jefa de Archivos, Registros y Habilitaciones (DARH)
Verificación documental y registro en sistema



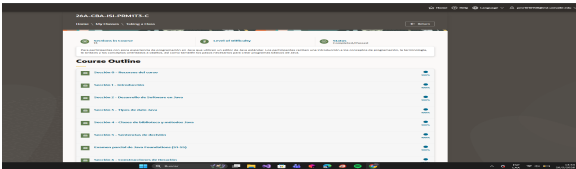
Firma

Dirección Académica
Verificación de asignaturas convalidadas



Firma

Secretaría (Vicerrectoría Académica)
Elaboración de Resolución Vicerrectoral



Firma

Vicerrector Académico
Firma de Resolución Vicerrectoral



Firma

Proceso Ágil SCRUM

1. Que es Scrum <https://proyectosagiles.org/que-es-scrum/>
2. Como funciona Scrum <https://proyectosagiles.org/como-funciona-scrum/>
3. Lista de objetivos / requisitos priorizada (Product Backlog)
<https://proyectosagiles.org/lista-requisitos-priorizada-product-backlog/>
4. Introducción a la estimación y planificación ágil
<https://proyectosagiles.org/2009/06/08/introduccion-estimacion-planificacion-agil/>
5. Estimación y planificación ágil
<https://proyectosagiles.org/2009/07/01/estimacion-planificacion-agil-quinto-encuentroagil-barcelona/>

ARTÍCULOS RELACIONADOS

6. Fundamentos de Scrum <https://proyectosagiles.org/fundamentos-de-scrum/>
7. Timebox <https://proyectosagiles.org/timebox/>

Qué es SCRUM

Scrum es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para **trabajar colaborativamente, en equipo**, y obtener el mejor resultado posible de un proyecto. Estas prácticas se apoyan unas a otras y su selección tiene origen en un estudio de la manera de trabajar de equipos altamente productivos.

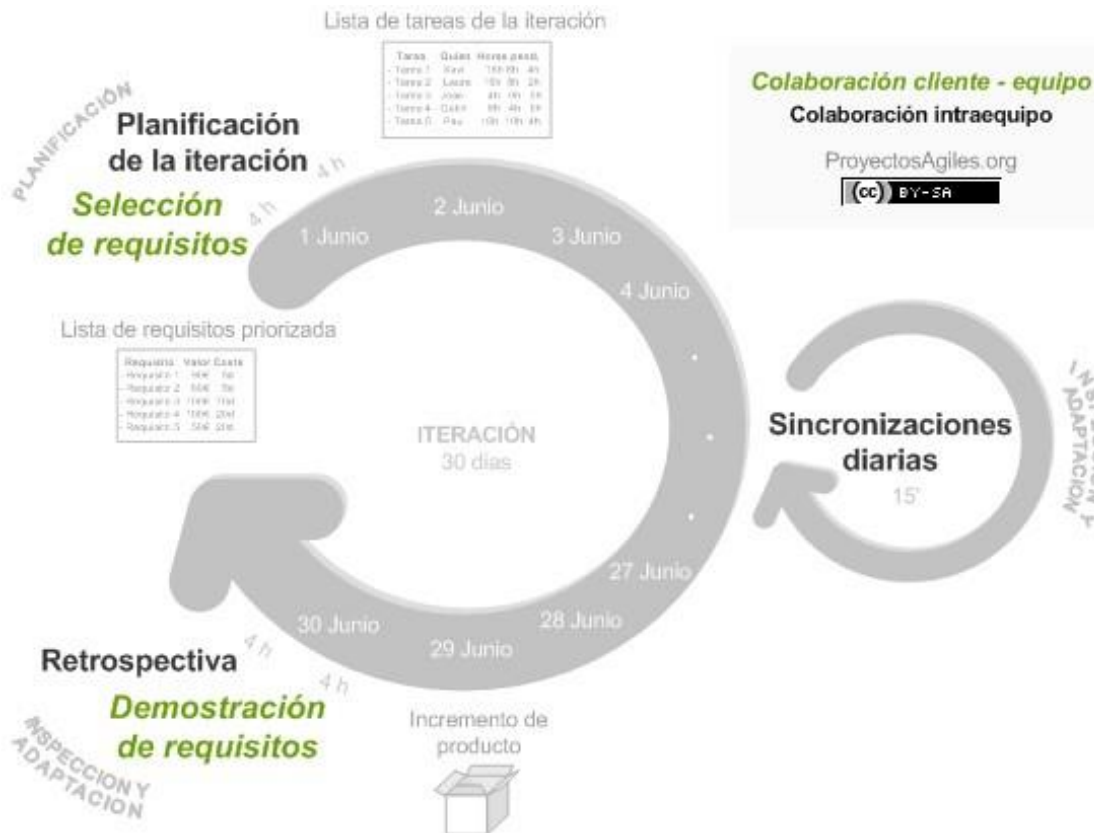
En Scrum se realizan entregas parciales y regulares del producto final, priorizadas por el beneficio que aportan al receptor del proyecto. Por ello, Scrum está especialmente indicado para proyectos en **entornos complejos**, donde se necesita **obtener resultados pronto**, donde los **requisitos son cambiantes o poco definidos**, donde la **innovación**, la **competitividad**, la **flexibilidad** y la **productividad** son fundamentales.

Scrum también se utiliza para resolver situaciones en que **no se está entregando al cliente lo que necesita**, cuando **las entregas se alargan demasiado**, **los costes se disparan** o **la calidad no es aceptable**, cuando se necesita **capacidad de reacción ante la competencia**, cuando **la moral de los equipos es baja y la rotación alta**, cuando es necesario **identificar y solucionar ineficiencias sistemáticamente** o cuando se quiere trabajar utilizando un **proceso especializado en el desarrollo de producto**.

Ver en detalle cuales son los beneficios de Scrum, sus fundamentos y sus requisitos.

El proceso

En Scrum un proyecto se ejecuta en ciclos temporales cortos y de duración fija (iteraciones que normalmente son de 2 semanas, aunque en algunos equipos son de 3 y hasta 4 semanas, límite máximo de feedback de producto real y reflexión). Cada iteración tiene que proporcionar un resultado completo, un incremento de producto final que sea susceptible de ser entregado con el mínimo esfuerzo al cliente cuando lo solicite.



El proceso parte de la lista de objetivos/requisitos priorizada del producto, que actúa como plan del proyecto. En esta lista **el cliente (Product Owner) prioriza los objetivos balanceando el valor que le aportan respecto a su coste** (que el equipo estima considerando la Definición de Hecho) y quedan repartidos en iteraciones y entregas.

Las actividades que se llevan a cabo en Scrum son las siguientes (**los tiempos indicados son para iteraciones de 2 semanas**):

Planificación de la iteración

El primer día de la iteración se realiza la reunión de planificación de la iteración. Tiene dos partes:

1. **Selección de requisitos** (2 horas). El cliente presenta al equipo la lista de requisitos priorizada del producto o proyecto. El equipo pregunta al cliente las dudas que surgen y selecciona los requisitos más prioritarios que prevé que podrá completar en la iteración, de manera que puedan ser entregados si el cliente lo solicita.
2. **Planificación de la iteración** (2 horas). El equipo elabora la lista de tareas de la iteración necesarias para desarrollar los requisitos seleccionados. La estimación de esfuerzo se hace de manera conjunta y los miembros del equipo se autoasignan las tareas, se autoorganizan para trabajar incluso en parejas (o

grupos mayores) con el fin de compartir conocimiento (creando un equipo más resiliente) o para resolver juntos objetivos especialmente complejos.

Ejecución de la iteración

Cada día el equipo realiza una **reunión de sincronización** (15 minutos), normalmente delante de un tablero físico o pizarra (Scrum Taskboard). El equipo inspecciona el trabajo que el resto está realizando (dependencias entre tareas, progreso hacia el objetivo de la iteración, obstáculos que pueden impedir este objetivo) para poder hacer las adaptaciones necesarias que permitan cumplir con la previsión de objetivos a mostrar al final de la iteración. En la reunión cada miembro del equipo responde a tres preguntas:

- ¿Qué he hecho desde la última reunión de sincronización para ayudar al equipo a cumplir su objetivo?
- ¿Qué voy a hacer a partir de este momento para ayudar al equipo a cumplir su objetivo?
- ¿Qué impedimentos tengo o voy a tener que nos impidan conseguir nuestro objetivo?

Durante la iteración el Facilitador (Scrum Master) se encarga de que el equipo pueda mantener el foco para cumplir con sus objetivos.

- Elimina los obstáculos que el equipo no puede resolver por sí mismo.
- Protege al equipo de interrupciones externas que puedan afectar el objetivo de la iteración o su productividad.

Durante la iteración, el cliente junto con el equipo refinan la lista de requisitos (para prepararlos para las siguientes iteraciones) y, si es necesario, cambian o replanifican los objetivos del proyecto (10%-15% del tiempo de la iteración) con el objetivo de maximizar la utilidad de lo que se desarrolla y el retorno de inversión.

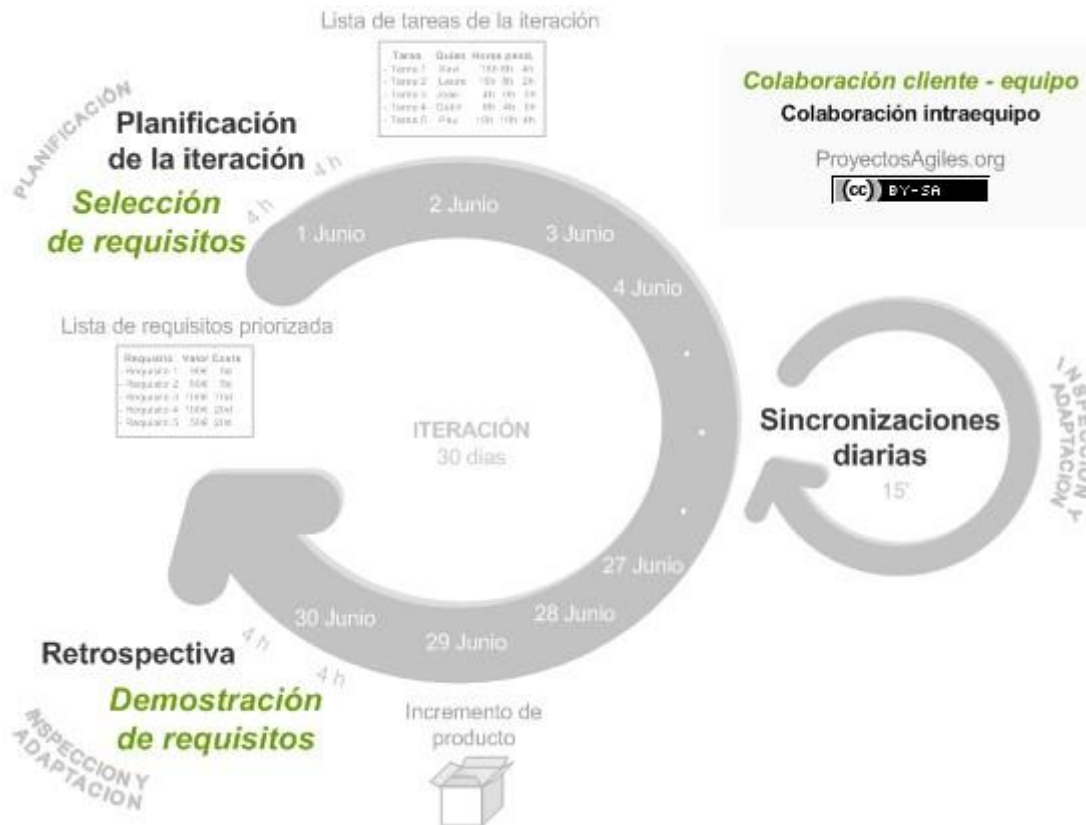
Inspección y adaptación

El último día de la iteración se realiza la reunión de revisión de la iteración. Tiene dos partes:

1. **Revisión (demostración)** (1,5 horas). El equipo presenta al cliente los requisitos completados en la iteración, en forma de incremento de producto preparado para ser entregado con el mínimo esfuerzo. En función de los resultados mostrados y de los cambios que haya habido en el contexto del proyecto, el cliente puede realizar las adaptaciones necesarias de manera objetiva, ya desde la primera iteración, replanificando el proyecto.
2. **Retrospectiva** (1,5 horas). El equipo analiza cómo ha sido su manera de trabajar y cuáles son los problemas que podrían impedirle progresar adecuadamente, mejorando de manera continua su productividad. El

Facilitador se encargará de eliminar o escalar los obstáculos identificados que estén más allá del ámbito de acción del equipo

Cómo funciona Scrum



Actividades

- Planificación de la iteración (Sprint Planning)
- Ejecución de la iteración (Sprint)
- Reunión diaria de sincronización del equipo (Scrum Daily Meeting)
- Demostración de los requisitos completados (Sprint Review)
- Retrospectiva (Sprint Retrospective)
- Refinamiento de la lista de requisitos y cambios en el proyecto

Responsabilidades

- Cliente (Product Owner)
- Facilitador (Scrum Master)

- Equipo de desarrollo (Team)

Herramientas

- Lista de requisitos priorizada (Product Backlog)
- Lista de tareas de la iteración (Sprint Backlog)
- Gráficos de trabajo pendiente (Burndown Chart)
- Definición de hecho (DoD)

Secciones relacionadas

- Beneficios de Scrum
- Fundamentos de Scrum
- Requisitos para poder utilizar Scrum

Herramientas que se suelen utilizar en Scrum

- Tablero de tareas
- Historias de usuario y criterios de aceptación
- Planning Poker
- Velocidad del equipo
- Principios de Lean Software Development

Conceptos y técnicas relacionadas

- Timebox
- Auto-organización y motivación intrínseca
- Skills en en equipo ágil
- Planificación ágil
- El expendedor – Juego de simulación de Scrum
- Base de conocimiento

Modelos mentales

- Modelo mental #1: AUTO-ORGANIZACIÓN / AUTONOMÍA Y MOTIVACIÓN.
- Modelo mental #2: GESTIÓN POR MIEDO VS CREAR UN ENTORNO SEGURO.
- Modelo mental #3: FEEDBACK AGRESIVO O PASIVO VS CLARIDAD, SINCERIDAD Y RESPETO.

Lista de objetivos / requisitos priorizada (Product Backlog)

La lista priorizada de objetivos/requisitos representa la visión y expectativas del cliente respecto a los objetivos y entregas del producto o proyecto

El orden de sus ítems viene determinado por el valor que aporta al cliente final respecto a riesgos y coste estimado de completarlo (ROI). Es una planificación estratégica que evoluciona a lo largo de toda la vida del producto/proyecto, debido a cambios de necesidades del cliente, feedback del mercado, aparición de nuevas ideas, dificultades tecnológicas, etc.

El cliente es el responsable de crear y gestionar la lista (con la ayuda del Facilitador y del equipo, quien proporciona el coste estimado de completar cada requisito). Dado que reflejar las expectativas del cliente, esta lista permite involucrarle en la dirección de los resultados del producto o proyecto.

- Contiene los objetivos/requisitos de alto nivel del producto o proyecto, que se suelen expresar en forma de historias de usuario. Para cada objetivo/requisito se indica el **valor** que aporta al cliente y el **coste estimado** de completarlo. La lista está priorizada balanceando el valor que cada requisito aporta al negocio frente al coste estimado que tiene su desarrollo, es decir, basándose en el Retorno de la Inversión (ROI).
- En la lista se indican las posibles **iteraciones** y las **entregas** (*releases*) esperadas por el cliente (los puntos en los cuales desea que se le entreguen los objetivos/requisitos completados hasta ese momento), en función de la velocidad de desarrollo del (los) equipo(s) que trabajará(n) en el proyecto. Es conveniente que el contenido de cada iteración tenga una coherencia, de manera que se reduzca el esfuerzo de completar todos sus objetivos.
- La lista también tiene que considerar los **riesgos** del proyecto e incluir los requisitos o tareas necesarios para mitigarlos.

Área de requisitos	Requisitos	Origen	Valor	Estimación inicial	Factor Ajuste	Estimación ajustada	Iteración:					
							Pendiente:	1	2	3	4	5
Área X	Requisito A	Marketing	2000	15		15	15	0				
Área Z	Requisito B	Producción	1750	20		20	20	0				
Área Y	Requisito C	Ventas	1500	20		20	20	0				
	Iteración 1		5250	55		55	55	0	0	0	0	0
Área Z	Requisito C	Producción	1250	15	0,2	18	18	18	0			
Área X	Requisito D	Producción	1250	20		20	20	20	0			
Área Z	Requisito E	Marketing	1000	15	0,2	18	18	18	0			
	Iteración 2		3500	50		56	56	56	0	0	0	0
	Primera entrega		8750	105		111	111	56	0	0	0	0
Área X	Requisito F	Marketing	1250	20	0,2	24	24	24	24	0		
Área Y	Requisito G	Marketing	750	15		15	15	15	15	0		
Área Y	Requisito H	Ventas	750	15	0,2	18	18	18	18	0		
	Iteración 3		2750	50		57	57	57	57	0	0	0
Área Z	Requisito I	Producción	700	15	0,2	18	18	18	18	18		
Área Y	Requisito J	Marketing	500	10	0,5	15	15	15	15	15		
Área Y	Requisito K	Ventas	500	20	0,2	24	24	24	24	24		
	Iteración 4		1700	45		57	57	57	57	57	0	0
	Segunda entrega		4450	95		114	114	114	114	57	0	0

Antes de iniciar la primera iteración, el cliente debe tener definida la **meta** del producto o proyecto y la lista de requisitos creada. **No es necesario que la lista sea completa ni que todos los requisitos estén detallados al mismo nivel.** Basta con que estén identificados y con suficiente detalle los requisitos más prioritarios con los que el equipo empezará a trabajar. Los requisitos de iteraciones futuras pueden ser mucho más amplios y generales. La incertidumbre y complejidad propia de un proyecto hacen conveniente no detallar todos los requisitos hasta que su desarrollo esté próximo. De esta manera, el esfuerzo de recoger, detallar y desarrollar el resto de requisitos (menos prioritarios) está repartido en el período de ejecución del proyecto. En el caso del desarrollo de un producto, la lista va evolucionando durante toda la vida del producto (los requisitos «emergen»). En el caso de un proyecto, conforme éste avance irán apareciendo los requisitos menos prioritarios que falten. Esto produce varias ventajas:

- Se evita caer en parálisis de análisis al inicio del proyecto, de manera que se puede iniciar antes el desarrollo y el cliente puede empezar a obtener resultados útiles.
- Se evita analizar en detalle requisitos no prioritarios que podrían cambiar durante el transcurso del proyecto, dado que el cliente conocerá mejor cuál ha de ser el resultado a conseguir, o bien por que podrían ser reemplazados por otros.
- Puede llegar a un punto del proyecto en que no valga la pena analizar ni desarrollar los requisitos restantes, dado el poco retorno de inversión (ROI) que tienen.

Iteración de entrega (release sprint)

Cuando el cliente solicita una entrega de los objetivos/requisitos completados hasta ese momento, el equipo puede necesitar añadir una **iteración de entrega**, más corta que las iteraciones habituales, donde

realizar alguna tarea que no ha sido necesaria o posible hasta el momento de la entrega final y acabar de corregir defectos detectados en la última demostración.

Idealmente **esta iteración de entrega no debería existir** (o reducirse a tareas mínimas dentro de una iteración) si:

- Se ha trabajado con una buena Definición de Hecho durante cada iteración del proyecto.
- Se hacen entregas muy cortas cada poco tiempo, con lo que la cantidad de cosas a entregar, integrar y probar es pequeña.
- Se está realizando un esfuerzo de automatización de estas tareas de pruebas, iteración y entrega durante todo el proyecto (con lo cual se gana en eficiencia y seguridad).

Uso de la lista de objetivos priorizada

- Para medir la velocidad de desarrollo del equipo, ver una progresión constante y extrapolar la fecha de las entregas, es conveniente seguir las siguientes recomendaciones:
- **Los requisitos deben tener un esfuerzo semejante** para ser completados.
- La estimación de un requisito **no debe ser superior a 10 días** (si las iteraciones son de 20 días laborables).
- **Cada requisito tiene asociado un factor de complejidad**, que permite ajustar su coste estimado en función de la incertidumbre de la complejidad de su desarrollo en el momento de introducirlo en la lista. Este factor de coste se irá ajustando conforme las iteraciones avancen y el equipo conozca mejor el producto o proyecto, su contexto y su velocidad de desarrollo con las herramientas y tecnologías que utiliza.
- **Si un requisito depende de otro, se coloca en algún punto por debajo del que depende.**
- Si un requisito no se finaliza en una iteración, se le vuelve a poner en alguna de las siguientes iteraciones, indicando el coste pendiente de desarrollo.
- El «origen» permite saber quién podría participar en la definición de un objetivo/requisito y sería conveniente que estuviese presente en el momento de su demostración.

El progreso del proyecto y su velocidad con respecto a requisitos completados se muestra en un gráfico de trabajo pendiente (Burndown chart).

Introducción a la estimación y planificación ágil

Autor: Xavier Quesada Allue

Saber estimar y planificar es fundamental a la hora de encarar proyectos donde el producto necesita de un grado importante de creatividad y/o innovación, como por ejemplo los de desarrollo de software. En este artículo, presentamos algunos principios y prácticas introductorias para aprender a estimar y planificar un proyecto ágil.

Una de las características de la gestión de proyectos ágiles es el ser una actividad adaptativa en vez de predictiva. No es extraño, entonces, que los procesos de estimación y planificación en un proyecto ágil sean radicalmente diferentes a los de un proyecto tradicional.

En un proyecto tradicional, el proceso es relativamente lineal: se estima el producto a desarrollar (generalmente haciendo un desglose por etapas); se planifica el desarrollo (con la consecuente transformación de lo que antes eran estimaciones en compromisos); y luego se procede a ejecutar el plan, que por supuesto debe cumplirse al pie de la letra. Cuando las cosas comienzan a atrasarse (y siempre lo hacen) empiezan las complicaciones.

El problema fundamental de la planificación tradicional es que trata al desarrollo de software como una actividad predecible, cuando no lo es. Y este problema fundamental es lo que intenta atacar la estimación y planificación ágil. **El desarrollo de software es una actividad de creación y transmutación de conocimiento.** Como tal, no puede ser predicha ni estimada en forma precisa. El primer paso hacia la planificación ágil es la aceptación de este concepto.

Pero pocas organizaciones están dispuestas a embarcarse en un proyecto sin tener siquiera una idea aproximada de cuánto va a costar o cuándo va a estar terminado el producto. Si esto fuera aceptado, podríamos dedicarnos directamente a producir sin ningún tipo de estimación o planificación (lo cual tal vez no sería mala idea).

Entonces, cómo encarar la estimación y planificación de algo que no sabemos predecir?

Bueno, empecemos por refinar un poco qué significa no poder predecir el tamaño del producto. En la práctica, cualquier desarrollador senior puede dar una idea del orden de magnitud de un proyecto. Esto nos brinda lo que en inglés se

denomina **ballpark figure**, un número grueso que nos permite ir pensando si es negocio desarrollar el producto o no. Y es lo primero que debe hacerse, ágil o no ágil. Las probabilidades de estar equivocados en un orden de magnitud son realmente bajas (en ese caso, por favor reconsiderar el título de «senior» de los desarrolladores). En mi experiencia, los proyectos tradicionales suelen excederse de sus estimaciones originales en numeros que van del 30% al 300%. Esto es lo que intentaremos mejorar con la técnica que explicaremos a continuación.

Las metodologías ágiles implementan muchos conceptos de Lean, el sistema de producción de Toyota. Uno de ellos es **small batch sizes**, que significa **producir valor en lotes pequeños**. El desarrollo tradicional, con sus etapas, produce todo el valor (el proyecto) en un solo lote. En todo momento, el 100% del proyecto está siendo procesado y 0% ha sido terminado. Finalmente se llega al «Día D», el «Big Bang», donde todo el proyecto es entregado de un saque. Los métodos ágiles, por contraste, buscan entregar valor incrementalmente. En el caso del desarrollo de software, esto se consigue agregando funcionalidad en cada iteración y manteniendo siempre el producto funcionando con la funcionalidad que haya sido implementada hasta ese momento.

Objetivos como historias de usuario

Siguiendo esta línea, el primer paso en la estimación y planificación ágil es la **creación del product backlog**, o sea la definición del proyecto a realizar. Se puede dividir en objetivos expresados como **historias de usuario** (user stories), cada una aportando valor de negocios incremental e individual. Una historia es un requisito de negocio visto desde el punto de vista de un usuario. Se escriben con el siguiente formato: «Como xxx, quiero hacer yyy con el objetivo de zzz», donde, xxx es el tipo de Usuario (quien), yyy es lo que el sistema debe permitir realizar (el qué) y zzz es el beneficio o valor buscado (el por qué).

Ejemplo:

«Como cliente del banco, quiero pedir un préstamo para poder comprar una casa»

.



Las condiciones de satisfacción de los objetivos suelen ponerse en forma de **criterios de aceptación**, pruebas que se realizarán para verificar si el sistema se comporta de la manera esperada. Para ello se puede utilizar la sintaxis de **BDD (Behaviour Driven Development)** siguiendo el siguiente formato: «*Dado aaa, cuando se produzca bbb, entonces ccc*», donde aaa es la situación en la que se encuentra el sistema, bbb es un evento que lo hará cambiar y ccc es el resultado. Esta técnica permite evitar la aparición de errores por malos entendidos y evitar retrabajar (siguiendo los principios Lean). Por ello es recomendable no empezar a desarrollar en una iteración sin antes haber escrito los casos de prueba, especialmente por que es más barato escribir texto y pensar en cómo desambiguar los requisitos que arreglar errores importantes debido a su mal entendimiento.

Pero en la práctica no hace falta usar estos formatos, cualquier sintaxis donde la acción sea clara y el beneficio buscado sea entendido por todos es suficiente. Si no partimos de cero, podemos simplemente tomar los requerimientos en cualquier formato que estén escritos (por ejemplo casos de uso).

Es importante no confundir Criterios de Aceptación de cada objetivo (requisito / historia de usuario) con la Definición de Hecho (DoD) que tienen que cumplir TODOS los requisitos.

Estimación con Planning Poker

El product backlog es, para ser exactos, una lista priorizada y estimada de historias. Por ahora sólo tenemos historias. Falta estimarlas y priorizarlas. El proceso de estimación se puede hacer utilizando una técnica llamada **planning poker** (póker de planificación). El objetivo del *planning poker* es obtener una medida de **tamaño relativo** de todas las historias respecto a sí mismas.



La teoría es que resulta relativamente fácil decir «A es más grande que B y que C» [no voy a entrar en detalle respecto a cómo efectuar *planning poker*, dejándolo para [otro artículo](#)]. Lo importante de efectuar *planning poker* sobre todo el backlog (a efectos de la planificación) es que da como resultado que todas las historias han sido estimadas con muy poco esfuerzo. Pero no en días/hombre como se haría tradicionalmente. *Planning poker* produce estimaciones en una medida arbitraria de tamaño llamada **story points** o «puntos de historia». Los *story points* son específicos de cada equipo, no pueden compararse entre diferentes equipos y a veces ni siquiera entre diferentes proyectos del mismo equipo. Lo único que indican es el tamaño relativo que tiene cada funcionalidad del backlog respecto a las demás. Lo importante es que ahora tenemos el tamaño total del proyecto estimado en una unidad llamada *story points*, y esto nos va a servir de mucho.

Priorización

La etapa de **priorización** es sencilla y depende exclusivamente del Product Owner. Sabiendo ya el tamaño de las historias, debe priorizarlas por **valor de negocio**. Notar que también es posible comenzar con la asignación de valor y después aportar el tamaño, en todo caso, la priorización se realiza balanceando el valor respecto al coste y respecto a los riesgos de cada objetivo.

Una manera rápida de empezar a asignar valor a las historias es dividir las en 3 grupos, según sean imperativas, importantes o cosméticas/ prescindibles (de manera que si se llega a una fecha de entrega predeterminada y no se han completado por lo menos hemos aportado el máximo de valor posible). Dentro de cada grupo nos resultará más fácil realizar una ordenación relativa por valor y después asignarlo.

La prioridad puede cambiar todo el tiempo; pero el tamaño en *story points* debe mantenerse fijo con la estimación original (o sea: como regla general, no reestimar). Si aparecen historias nuevas, deben estimarse utilizando el mismo criterio que se utilizó originalmente.

Ahora bien: todo esto todavía no nos dice nada respecto a cuánto durará o costará el proyecto; pero al menos es un paso más respecto a como estábamos antes, que solo teníamos el *ballpark estimate*. Si sólo pudiéramos averiguar a **cuántos días/hombre o días/equipo equivale un *story point***, tendríamos nuestra estimación, y luego nuestra planificación.

Duración y proyección a partir de la velocidad del equipo

El último paso, por lo tanto, es calcular la **velocidad del equipo** completando objetivos a lo largo de las iteraciones. Así pues, la velocidad es **la cantidad de *story points* que se**

completan por iteración. Calcularla es sencilla: solo hay que sentarse y esperar. En dos -como máximo tres- iteraciones, tendrás una idea bastante clara de cuál es la velocidad del equipo y por lo tanto el tamaño y duración del proyecto. Mientras tanto se puede ir construyendo el **burndown chart**, cosa que no me animo a traducir (gráfico de quemado?). El *burndown chart* nos muestra en el eje Y la cantidad total de story points del proyecto, y sobre el eje X las iteraciones. Cada vez que se finaliza una iteración, se completa un punto del gráfico, indicando la velocidad en ese ciclo.

Si teníamos una fecha prefijada en la que queremos terminar el proyecto, esto nos permite calcular la velocidad teórica a la que tendremos que ir para alcanzar esa fecha. El **burndown chart** permite rápidamente y en todo momento ver dos estadísticas vitales para la planificación: la estimación actual de **cuándo va a estar terminado el 100% del proyecto**; y la estimación del **porcentaje de proyecto que va a estar terminado cuando lleguemos a cierta fecha**.

Conclusión

La estimación y planificación ágil permiten así en todo momento **saber cuál es la fecha estimada de finalización del proyecto, y en qué iteración estará lista determinada funcionalidad**. Un beneficio adicional que nos brinda es que **de existir complicaciones severas**, que pongan en juego la factibilidad del proyecto, éstas generalmente **se ven expuestas bien temprano**, permitiendo cancelar el proyecto antes de incurrir en grandes pérdidas. Por esto, sumado al hecho de que **el desarrollo iterativo e incremental garantiza que en todo momento se cuenta con el producto listo para ser entregado** (por ejemplo software funcionado), está el hecho de que **los métodos ágiles disminuyen enormemente los riesgos tradicionales en el desarrollo de proyectos**.

COMENTARIOS

La clave al estimar en story points es olvidarse de los días hombre por un rato. Esto nos permite pensar en la complejidad del problema, en vez de perdernos en detalles de implementación y cosas propias del equipo.

Para empezar, se toma una historia que, a ojo, parezca una de las más pequeñas del backlog. A esa historia le damos tamaño un story point. (Algunos prefieren darle tamaño dos story points, para dejar lugar a que aparezca algo más pequeño más adelante.) Luego se procede a estimar todo el backlog secuencialmente, usando esta escala. Cada historia se compara con las anteriores -ya estimadas- para orientarse respecto a su tamaño.

El cálculo de Velocidad se puede hacer -desde ya- desde la primera iteración, pero al no haber ningún tipo de información histórica, tiene tanto valor como una adivinanza. En general uno toma el total de story points que el equipo decidió podía completar como referente a lo que el equipo piensa que va a ser su velocidad. Siempre medido en story points. Luego, al concluir la iteración, se mide

el número de story points que fueron realmente entregados (demostrados y aceptados por el Product Owner).

Sabiendo el número de story points terminados, más la cantidad de días hombre que fueron “gastados” en la iteración, es tarea sencilla hacer la conversión entre “story points” y “días/hombre”. Tener en cuenta que este número puede variar más adelante, a medida que el equipo mejora (por ejemplo al rendir fruto sus retrospectivas) o empeora (por ejemplo al acumular deuda técnica)

Estimación y planificación ágil – Resultados del quinto encuentro ágil en Barcelona

En este encuentro se compartieron experiencias sobre los siguientes temas:

- Gestión de dependencias entre objetivos de proyecto.
- Estimación del proyecto por parte de todo el equipo, utilizando días ideales o bien puntos de historia de usuario.
- Planning Poker para hacer una estimación inicial del proyecto rápida y fiable.
- Uso de la velocidad de desarrollo para ir proyectando el final del proyecto.
- Pruebas de concepto para realizar estimaciones.
- Estimación y planificación de iteración basada en compromiso.
- La estimación ágil ayuda a crear “conciencia de equipo”



Estimación y planificación de proyecto

Dependencia entre objetivos del proyecto

- **Los objetivos** (en forma de historias de usuario) que forman la lista de objetivos priorizada del proyecto (Product Backlog) **deben ser lo más independientes posibles**, para poder demostrarlas y enseñarlas al cliente con la seguridad de que están completadas. De esta manera, el cliente puede tomar decisiones objetivas basándose en los resultados que ya está obteniendo del proyecto y en la velocidad de desarrollo del equipo.
- Si hay objetivos que son muy dependientes, se puede hacer varias cosas:
- El objetivo que genera más dependencias se pone antes en el Product Backlog. Los objetivos dependientes se sitúan en algún punto por detrás.
- En alguna situación puede ser interesante juntar varios objetivos dependientes en uno. Sin embargo, con objetivos excesivamente grandes es más difícil medir el progreso del proyecto y cuesta más tiempo completarlos, por lo que si hay un retraso se tarda más en que sea visible. Por ello, para observar un avance regular en el proyecto, es mejor que los objetivos tengan un tamaño similar y no sean demasiado grandes.

La estimación la lleva a cabo el equipo

- La estimación no la realiza el cliente / Product Owner, por que no es él quien se va a "ensuciar las manos" y luchar por cumplir con fechas.
- Todo el equipo realiza la estimación para:
- Reforzar el compromiso de todo el equipo respecto a las fechas que dan al cliente.
- Reforzar el compromiso de cada miembro del equipo respecto al resto.
- Hacer que todo el mundo se sienta oído.
- La estimación se puede realizar utilizando alguna de las siguientes unidades:
- **Días ideales**: los días necesarios para que el equipo pueda completar un objetivo, sin considerar interrupciones. Para pasar a días reales hay que aplicar un factor de corrección que puede ir del 60 % al 70 % de dedicación real al proyecto. Asimismo, habrá que tener en cuenta un margen para imprevistos como bajas por enfermedad, etc.
- **Puntos de historia de usuario**: la complejidad que tiene cada historia de usuario. Un equipo en un proyecto determinado es capaz de completar un número semi-regular de puntos de historia cada iteración ("velocidad").

Planning poker

La técnica de planning poker permite hacer una **estimación inicial del proyecto rápida y fiable**, dado que todos los miembros del equipo comparten sus diferentes informaciones y expresan su opinión sin sentirse condicionados por el resto.

A continuación se puede ver diferentes barajas de cartas de planning poker.



Cada número significa un peso / esfuerzo / complejidad para completar un objetivo (historia de usuario). La numeración de las cartas está basada en la sucesión de Fibonacci. La distancia entre números crece conforme se hacen mayores. De esta manera, se facilita la decisión sobre qué tamaño tiene un objetivo.

¿Es un 5 o un 8?. No vale la pena entretenerse en pensar si es un 5 o un 6, ni en el error por no intentar buscar esta precisión, dado que se compensará por encima y por debajo con el resto de estimaciones.



José Raya, Jordi Pradel y Alexis Roqué haciendo una demostración de Planning Poker

Planning Poker es un proceso iterativo de planificación. Funciona de la siguiente manera:

- El cliente lee un objetivo (historia de usuario escrita en una tarjeta).
- El equipo le hace preguntas para entender su alcance.
- Las respuestas importantes se pueden apuntar en la propia tarjeta como detalles del objetivo o condiciones de satisfacción.
- Pueden aparecer nuevas historias de usuario.
- Cada miembro del equipo piensa en el esfuerzo necesario para completar el objetivo y todos muestran sus tarjetas simultáneamente, de manera que no están condicionados por las estimaciones de los otros.
- Las personas que están más alejadas del consenso explican por qué su votación es más alta (hay algún problema en el que nadie más ha pensado o el resto no ha tenido en suficiente consideración) o más baja (conocen una manera sencilla de resolver el problema, resolvieron algo muy parecido en un proyecto anterior, etc.).
- El equipo vuelve a votar, hasta que alcanza un acuerdo. No hay democracia, dado que todos deberán comprometerse a que ese objetivo se va a acabar con el esfuerzo acordado (se supone que no hay una persona que siempre vota de manera singular y a la que nunca se puede convencer).

Diversas personas manifestaron que con esta técnica el equipo ha ido mejorando mucho la precisión de sus estimaciones. De hecho, existen diversos estudios que concluyen que la sinergia que produce esta estimación conjunta es mucho mejor que la de una única persona por separado (típicamente la del jefe de proyecto o un senior en un proyecto tradicional).

Algunos consejos y trucos:

- Elegir un objetivo de tamaño típico en el proyecto como patrón con el que comparar el resto de objetivos y asignarle una puntuación de carta también media (por ejemplo un 5).
- Puede ser conveniente no jugar con las cartas más altas, ya que el error de estimación es mucho mayor (además, objetivos tan grandes dificultan ver el progreso y se tarda más en hacer visible si existen problemas en el proyecto). Notar que en la fotografía de las barajas se han apartado las cartas más altas para no jugar con ellas.
- Dada la dificultad de empezar a trabajar con puntos de historia y no con esfuerzo en días ideales, para facilitar el empezar a utilizar Planning Poker se puede hacer el símil de que, por ejemplo, dos puntos de historia corresponden a un día ideal.
- Comparar el tamaño que se va dando a cada objetivo respecto al de otros (triangulación). Ver ejemplo en [2].

Velocidad de desarrollo

Los puntos de historia permitirán medir la **velocidad** que tiene el equipo completando objetivos a lo largo de las iteraciones (puntos de historia por iteración) y de esta manera ir proyectando el final del proyecto.

- Las 2-3 primeras iteraciones la velocidad es más inestable (debido a la complejidad propia de un proyecto: dominio/ requisitos nuevos, tecnología nueva, personas nuevas) y después se va estabilizando, con lo que ya es posible utilizarla para hacer predicciones. Si no se estabiliza, posiblemente es por que existen problemas subyacentes que lo impiden (en la organización, en la estabilidad del equipo, etc.) y que habrá que solucionar.
- Las estimaciones (y la velocidad) se ven afectadas cuando se introducen nuevas tecnologías o conceptos de negocio en medio del proyecto, aunque después se vuelven a estabilizar.

Pruebas de concepto para realizar estimaciones

- Si existe incertidumbre para realizar una estimación de un tipo de objetivos (historias de usuario), se puede hacer un ***spike* o prueba de concepto extremo a extremo** en una iteración anterior a su desarrollo. Esta investigación permite conocer el grado de dificultad de completar algo y poder estimar mejor las siguientes iteraciones, donde se desarrollarán las historias de usuario reales.
- La prueba de concepto debe realizarse dentro de un *timebox*, para obligar a tomar decisiones y no dedicar un tiempo excesivamente largo a investigar.

Estimación y planificación de iteración basada en compromiso

En la reunión de planificación de iteración (Sprint Planning) el equipo va seleccionando objetivos del Product Backlog (historias de usuario) y descomponiéndolos en tareas. Escoge tantos objetivos como cree que es capaz de completar en una iteración (*commitment driven sprint planning*). La velocidad permite verificar (*checkpoint*) que el total de puntos de historia de la nueva iteración es congruente con las anteriores iteraciones, si están escogiendo demasiados objetivos o demasiado pocos.

Cada miembro del equipo que escoge una tarea hace su estimación en horas delante de todos, de manera que:

- Se compromete respecto a sus compañeros.
- Evita estimaciones demasiado optimistas (que comprometerían las fechas del proyecto) o demasiado pesimistas (que atentarían contra la competitividad de la empresa). De esta manera se disminuyen y ajustan los buffers.

La estimación ágil ayuda a crear “conciencia de equipo”

Como consecuencia de las técnicas descritas, la estimación ágil crea conciencia de equipo. Todos participan, tienen voz y voto. Cuando se produce una equivocación en una estimación, se ha equivocado todo el equipo. Por ello, no es para individualistas. No sirve que alguien diga “Yo he hecho mi parte de la historia de usuario”.